

Software Testing and Quality Assurance

Lecture 10

Fabian Fagerholm

fabian.fagerholm@aalto.fi



Aalto University
School of Science



Week	Lecture	Topic	Assignment deadlines (Tuesdays 10:00 unless otherwise specified)
36	3.9.2024	Introduction and practicalities	
37	10.9.2024	Software quality	Individual assignments 1
38	17.9.2024	Software testing: levels, test case analysis and design	Group registration (DL: 20.9.)
39	24.9.2024	Testing techniques: Black-box, white-box testing	Individual assignments 2, Group assignment 1
40	1.10.2024	Testing techniques: Manual testing	Individual assignments 3
41	8.10.2024	Testing techniques: Black-box, white-box, manual testing	Individual assignments 4
42	15.10.2024	(No lecture)	
43	22.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 5
44	29.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 6
45	5.11.2024	Guest lecture	Individual assignments 7, Group assignment 2
46	12.11.2024	Continuous Integration and Continuous Delivery/Deployment, Test management, and Course summary	Individual assignments 8
47	19.11.2024	(No lecture)	Individual assignments 9
48	26.11.2024	(No lecture)	Individual assignments 10, Group assignment 3



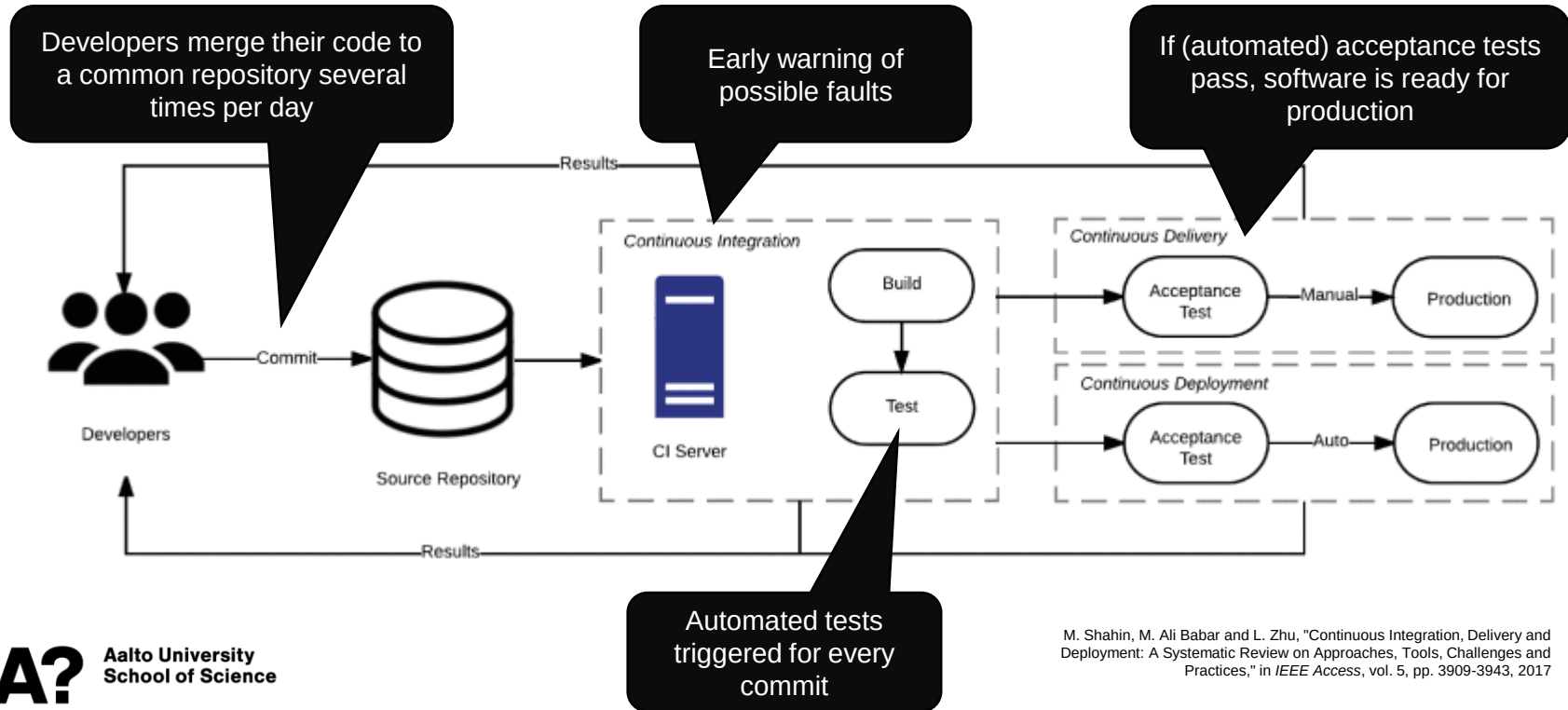
Continuous software engineering

- Conventional ways to manage testing
 - Full requirements & specifications up front
 - **Test-last: testing is performed after implementation**
 - Stricter division between testing and development personnel
 - Challenge: Time until testing is long → may have to spend lot of effort to fix, causing delays
- Agile / continuous ways to manage testing
 - Only a sprintful of requirements & specifications at a time
 - **Test-first: tests are defined with the requirements and drive development**
 - Cross-functional teams: close collaboration between testers and developers
 - Challenge: Testing fast enough as the test suite grows
 - Often supported by test automation



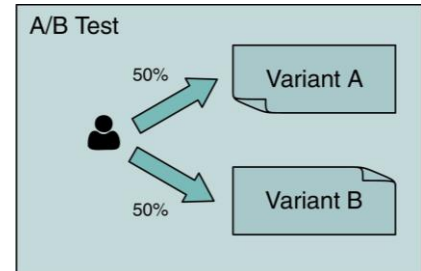
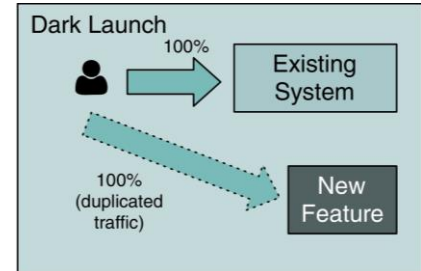
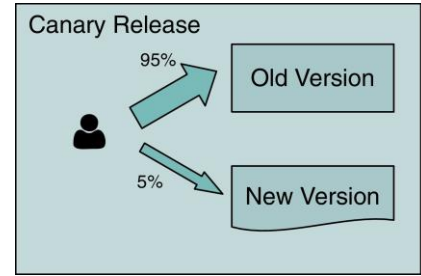
Picture licensed under [CC BY-SA-NC](#)

Continuous Integration – Delivery – Deployment



CI/CD enables several continuous software engineering practices

- Canary releases
 - Deploy new version of software to a small percentage of users
 - If there are failures, roll back
 - If everything is ok, deploy to all users
- Gradual roll-out
 - Like canary releases but deploy to more and more users
- Continuous Experimentation
 - Run field tests with real users to try how they react to new features
 - Allows finding causal relationships between features and user behaviour
 - Simple, e.g., A/B testing
 - More complicated, e.g., multi-armed bandit methods
 - Example: Which leads to more playing in a platform game?
 - A) Having the character return to the start when they die.
 - B) Having the character continue from the place they died.
 - Experiment: A/B test with 10% of players each, measure play time.



Managing software testing

Metrics for software quality

Product metrics

- Describe the characteristics of the product (size, complexity, quality level, ...)

Process metrics

- Can be used to improve software development and maintenance (effectiveness of defect removal, pattern of testing defect arrival, response time of the fix process)

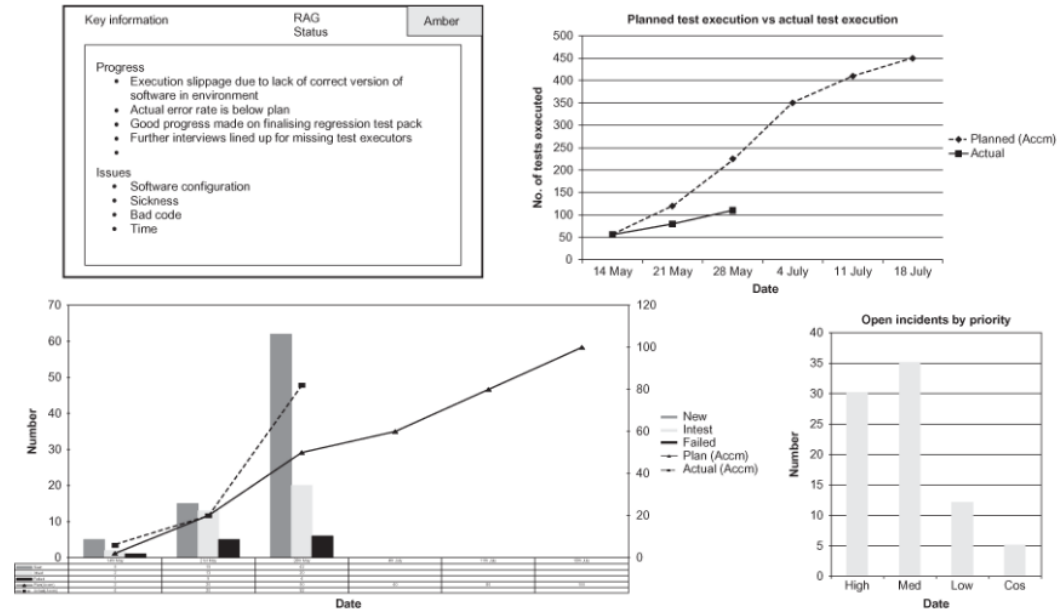
Project metrics

- Describe project characteristics and execution (number of software developers, cost, schedule, ...)

Monitoring progress

- Metrics allow us to monitor how the test plan is executed
- Feedback and visibility on how the testing is progressing
- Allows prioritizing testing effort
- Can also include monitoring of operations metrics

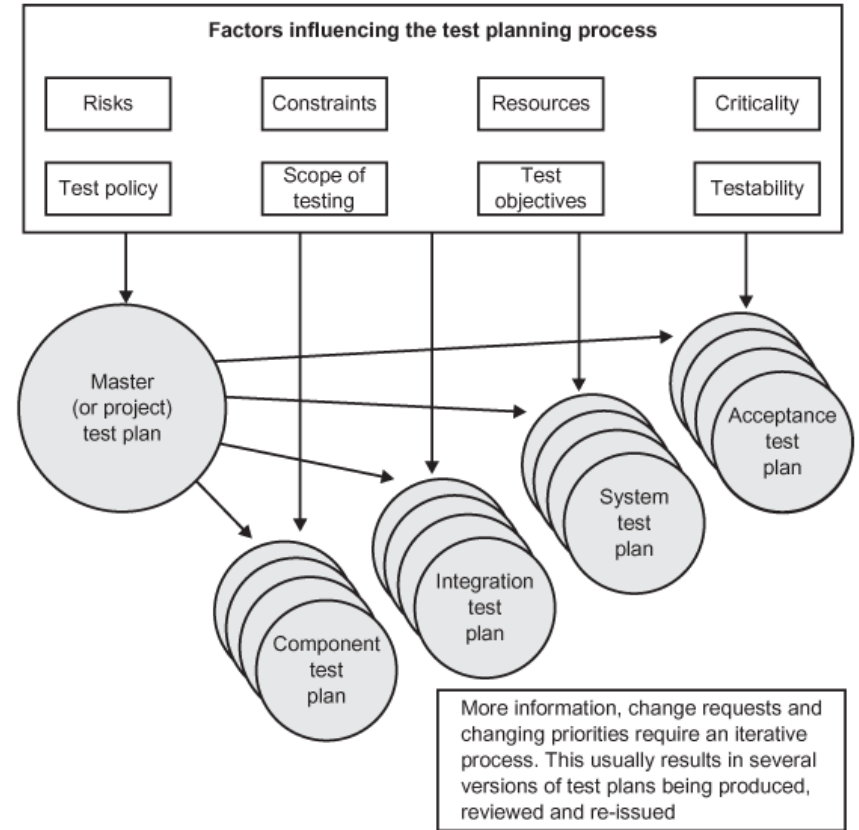
Figure 5.3 iTesting Executive Dashboard



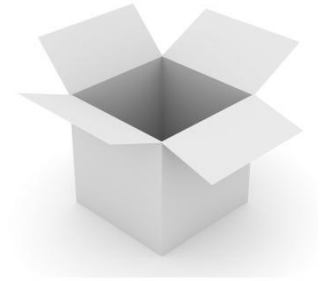
Test plans

- Steer the testing
 - What to test (scope)
 - How to test it (scope, objectives)
 - When to test it (schedule)
 - When will testing finish (effort, schedule)
 - Who does the testing (resource)
- Tasks or milestones for testing
- Track progress
- Define the direction and size of the effort
- In iterative development (e.g. agile), the plan will be continuously updated

Figure 5.2 Test plans in the V-model

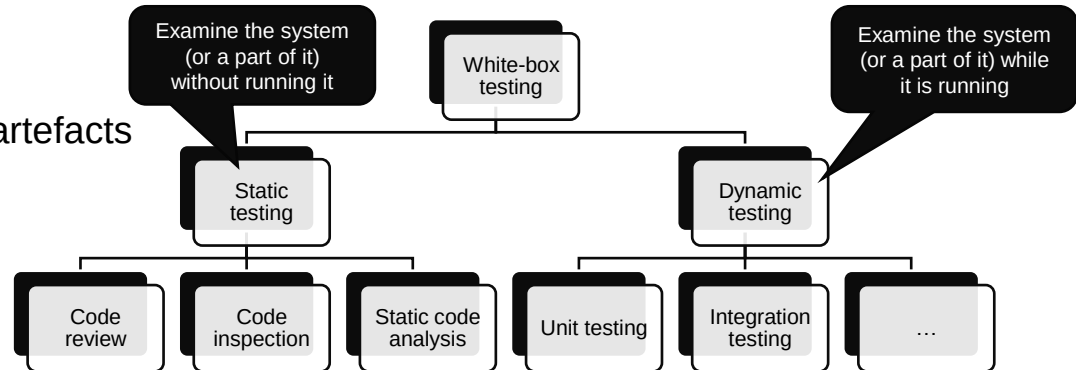


Reviews and inspections



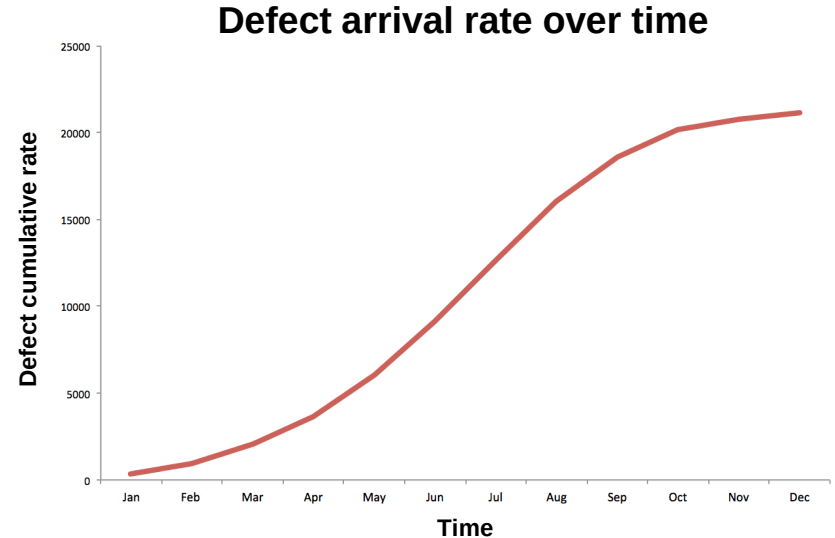
[Photo CC BY-SA-NC](#)

- Code reviews
 - Developers walk through code and try to find faults and (often) improve maintainability
 - Can be done asynchronously
- Inspections
 - Formal sessions
 - Uses a predefined protocol
 - Can be used to test non-code artefacts



Managing the testing process

- Testing usually results in more reported faults
- Development usually results in more faults in the SUT
- By monitoring the **defect arrival rate**, we can estimate how many faults are likely left in the software
- This allows us to make informed choices about the readiness of the software (e.g., for release decisions)



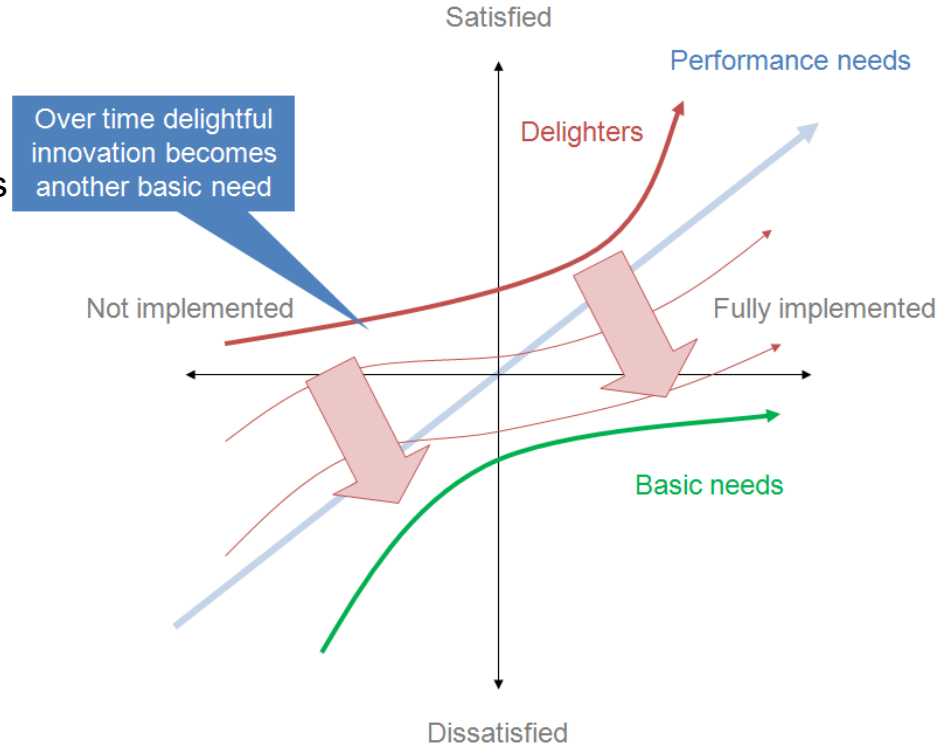
Discussion

How much quality is enough?
(Does your software have to be “perfect”?)

How much quality is enough?

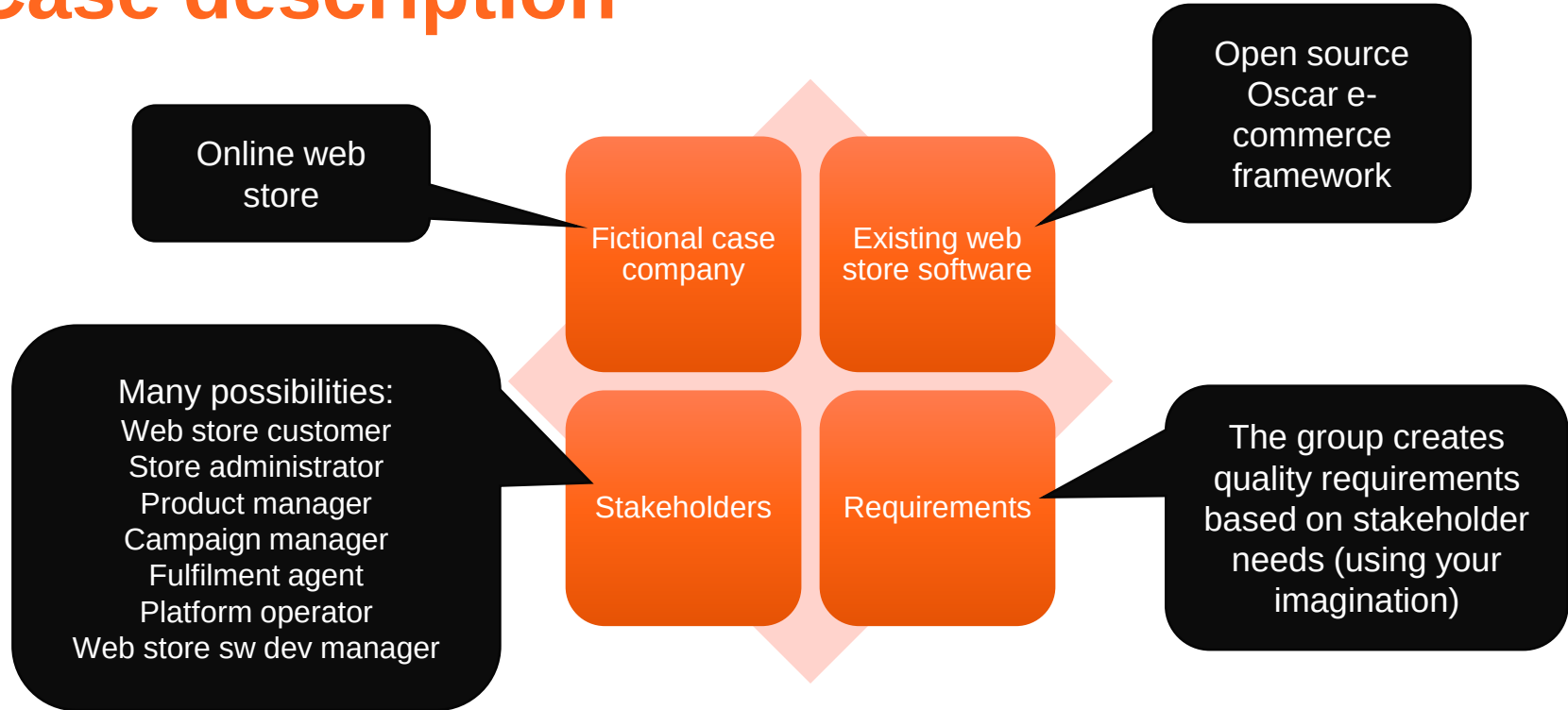
- Kano model
 - A theory for product development and customer satisfaction
 - Emphasises that customers' expectations change over time

→ Need to reassess stakeholder expectations from time to time



Group assignment: Final report

Case description



Tasks and deliverables

Prepare

- Read the project case description
- Choose two stakeholders to focus on
- *Document as report introduction*

Create a quality model

- Analyse stakeholder needs
- Create a quality model according to ISO 25010
- *Document as a table + definitions*

Define quality control activities

- Tests
- Inspections
- Reviews
- Metrics
- Other appropriate quality control activities
- *Document as table + text*

Define a test plan

- At least 10 test cases (of which at least 1 automated unit test case, 1 automated acceptance test case, 1 manual test case)
- *Document with inputs and expected outputs*

Execute a few tests

- Results from running the tests
- You can reuse tests that you have already carried out in the course – but they must be properly documented
- *Document test results*

Make quality assessment

- What is the current quality of the SUT for different stakeholders?
- *Document as text*

Final deliverable: a single PDF. A report template is provided in the assignment

Next steps

Use the related reading
for more details.
Reserve enough time to
set up your environment.

- Individual assignments in MyCourses
 - CD/CD 1: Project setup (DL: 19.11)
 - Test management (DL: 19.11)
 - CD/CD2: Pipeline configuration and execution (DL: 26.11)
- Group assignments in MyCourses
 - Final report (DL 26.11)
 - Peer review (DL 3.12)
- Getting help
 - Ask in the course chat, see Communication section in MyCourses
 - Personal questions by email



Study smarter, not harder!

- Plan: when and where
- Go deep at your own pace
- Get some rest
- Practice makes perfect
- Enjoy it ☺

Group summary: Interactive activity



Aalto University
School of Science